Handbook 01

Handling Memory Leaks in React app

A Guide to Best Practices and Optimization Techniques, with quick code Snippets

@arkajitroy

@arkajitroy

Introduction

Why Handling Memory Leaks Matters

Memory leaks in React applications occur when unused resources (like event listeners, subscriptions, timers, or references) aren't properly cleaned up. Over time, this causes performance degradation, unnecessary memory consumption, and even browser crashes.

In production-grade applications, ignoring memory leaks can result in:

- Gradual slowdown during user sessions
- High CPU usage due to unmounted component references
- Increased memory footprint impacting mobile users

Understanding how to prevent memory leaks is essential for building scalable, maintainable, and user-friendly React systems.

@arkajitroy

Common Pitfalls

Common Causes of Memory Leaks in React

Memory leaks typically occur when components don't release resources after unmounting. Some common causes include:

1. **Uncleared Timers/Intervals -** setInterval or setTimeout not being cleared.
2. **Unsubscribed Listeners –** event listeners remaining active after component unmount.
3. **Unfinished Async Calls –** API promises updating state after component unmount.
4. **Improper useEffect cleanup –** side effects not being cleaned up correctly.

```
useEffect(() => {
  const interval = setInterval(() => {
    console.log("Running safely...");
  }, 1000);

  return () => clearInterval(interval); // ✅ Cleanup
}, []);
```

@arkajitroy

Common Pitfalls

Clean Up Observers & External Libraries

Always clean up observers or external libraries when a component unmounts.

Tools like **IntersectionObserver**, **MutationObserver**, or **map/3D libraries (Mapbox, Three.js)** keep references alive if not disconnected, causing memory leaks.

Simple rule: if you attach it, you must detach it.

```
useEffect(() => {
  const observer = new IntersectionObserver(([entry]) => {
    if (entry.isIntersecting) console.log("Visible");
  });
  observer.observe(ref.current!);

  return () => observer.disconnect(); // cleanup
}, []);
```

@arkajitroy

Common Pitfalls

Handling Async Calls Safely

A common memory leak occurs when an async API call updates state after a component has unmounted. Use an abort controller or a mounted flag to prevent state updates.

This prevents React from trying to update state on an unmounted component.

```
useEffect(() => {
  const controller = new AbortController();

  fetch("/api/data", { signal: controller.signal })
    .then(res => res.json())
    .then(data => {/* handle */})
    .catch(err => {
      if (err.name !== "AbortError") console.error(err);
    });

  return () => controller.abort(); // cleanup
}, []);
```

@arkajitroy

Common Pitfalls

Event Listeners Cleanup

Global listeners, like window.addEventListener, should always be removed during cleanup. Neglecting this can cause major leaks when navigating between pages.

This pattern ensures components don’t leave dangling listeners after unmount.

```
useEffect(() => {
  const controller = new AbortController();

  fetch("/api/data", { signal: controller.signal })
    .then(res => res.json())
    .then(data => {/* handle */})
    .catch(err => {
      if (err.name !== "AbortError") console.error(err);
    });

  return () => controller.abort(); // cleanup
}, []);
```

@arkajitroy

Best Practice

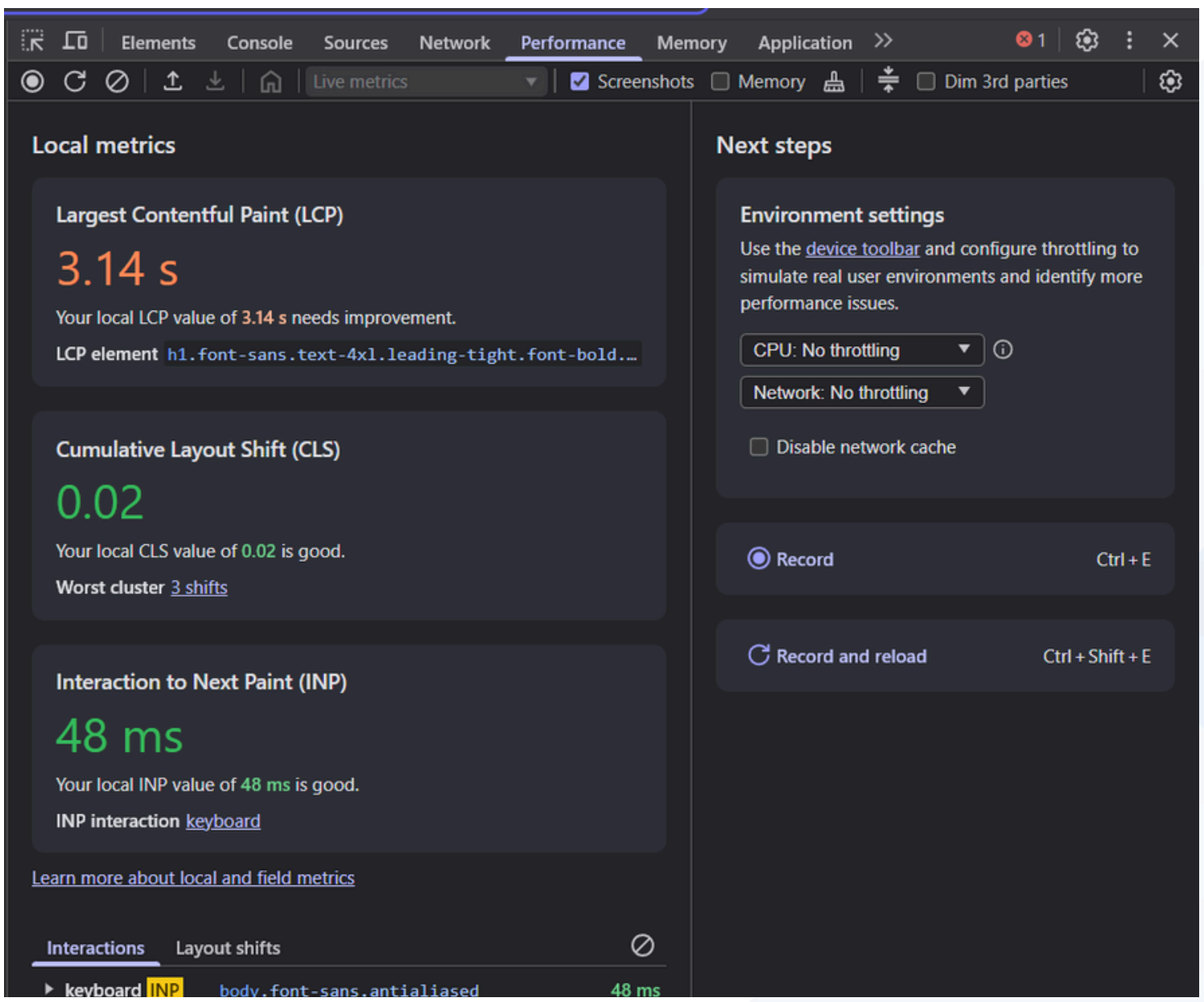
Detecting and Debugging

Use Chrome DevTools or React Profiler to identify leaks.

Steps:

1. Open Performance Tab → Record session.
2. Look for continuous memory growth over interactions.
3. Check unmounted components that still consume memory.

In large applications, monitoring tools (New Relic, Datadog, Sentry) can automate leak detection and alert developers.



@arkajitroy

Summarizing

Quick Checklist for Teams

- Every effect has a cleanup (event listeners, intervals, subscriptions).
- Fetch/async calls have an abort mechanism.
- Don’t store heavy objects in state/context unnecessarily.
- Release external library instances on unmount.
- Test with React StrictMode and watch for warnings.
- Run heap snapshots on critical flows (mount/unmount cycles).

Handy Monitoring Tools

- React Strict Mode: Helps detect unexpected side effects.
- DevTools Profiler: Shows retained memory.
- Heap snapshots in Chrome DevTools: Identify objects that persist unexpectedly.
- **ESLint rule:** Use eslint-plugin-react-hooks to enforce correct effect dependencies.

Thanks for the read!

Follow for more such contents

arkajitroy.work@gmail.com